MACHINE LEARNING

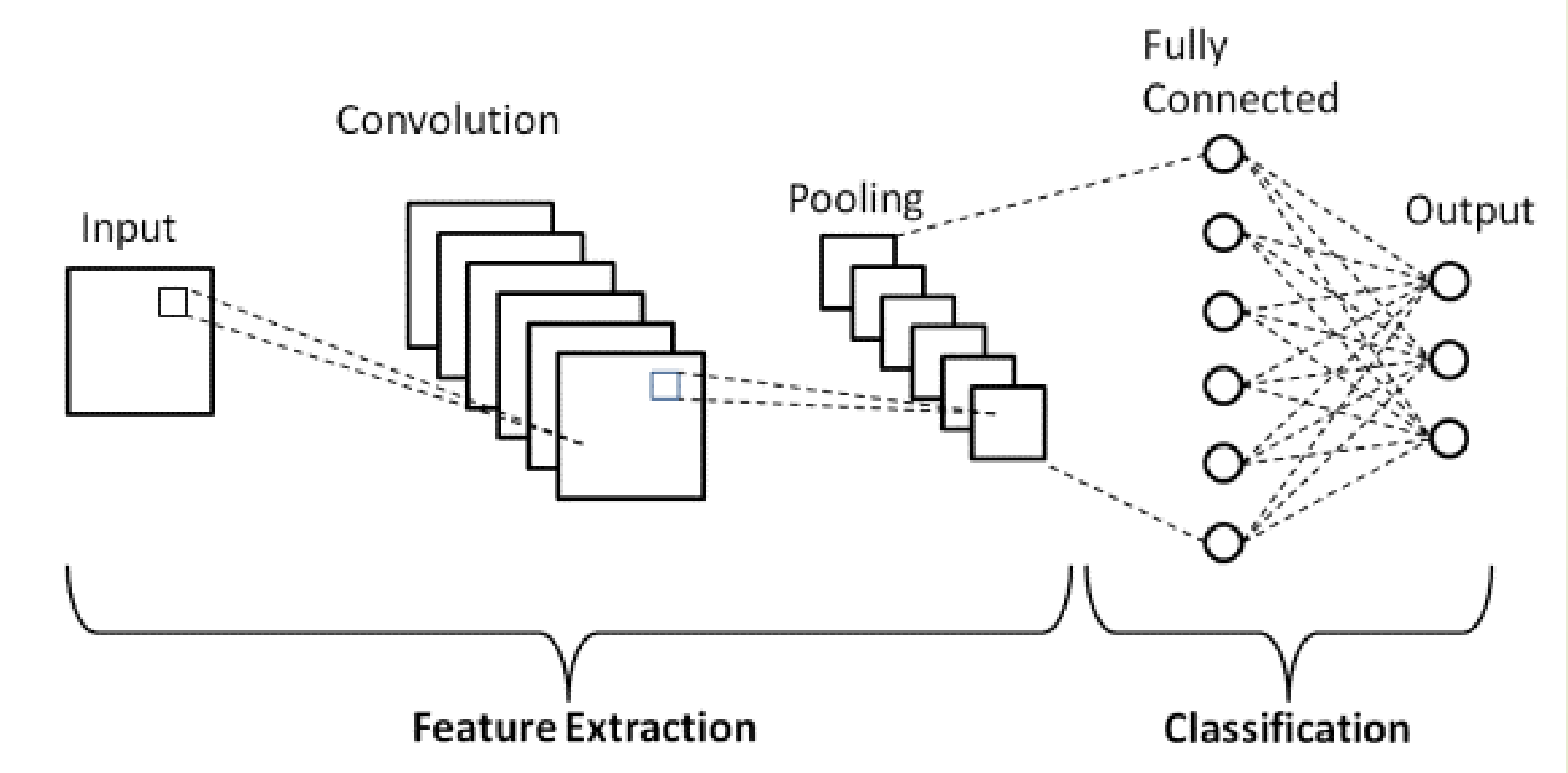
Convolutional Neural Network

A brief history of convolutional neural networks

- Convolutional neural networks, also called ConvNets, were first introduced in the 1980s by Yann LeCun, a postdoctoral computer science researcher.
- The early version of CNNs, called LeNet (after LeCun), could recognize handwritten digits. CNNs found a niche market in banking and postal services, where they read zip codes on envelopes and digits on checks.
- But, ConvNets remained on the sidelines of computer vision and artificial intelligence because they faced a serious problem: They could not scale.
- CNNs needed a lot of data and compute resources to work efficiently for large images.

- LeCun would continue his research with his team throughout the 1990s, culminating with "LeNet-5", which applied the same principles of prior research to document recognition. Since then, a number of variant CNN architectures have emerged with the introduction of new datasets, such as MNIST and CIFAR-10, and competitions, like ImageNet Large Scale Visual Recognition Challenge (ILSVRC).
- Some of these other architectures include:
- AlexNet
- VGGNet
- GøogLeNet
- ResNet
- ZFNet
- However, LeNet-5 is known as the classic CNN architecture.

CNN-Phases


Fully
Connected
Convolution
Pooling
Input
Output
Feature Extraction
Classification

Two phases :-

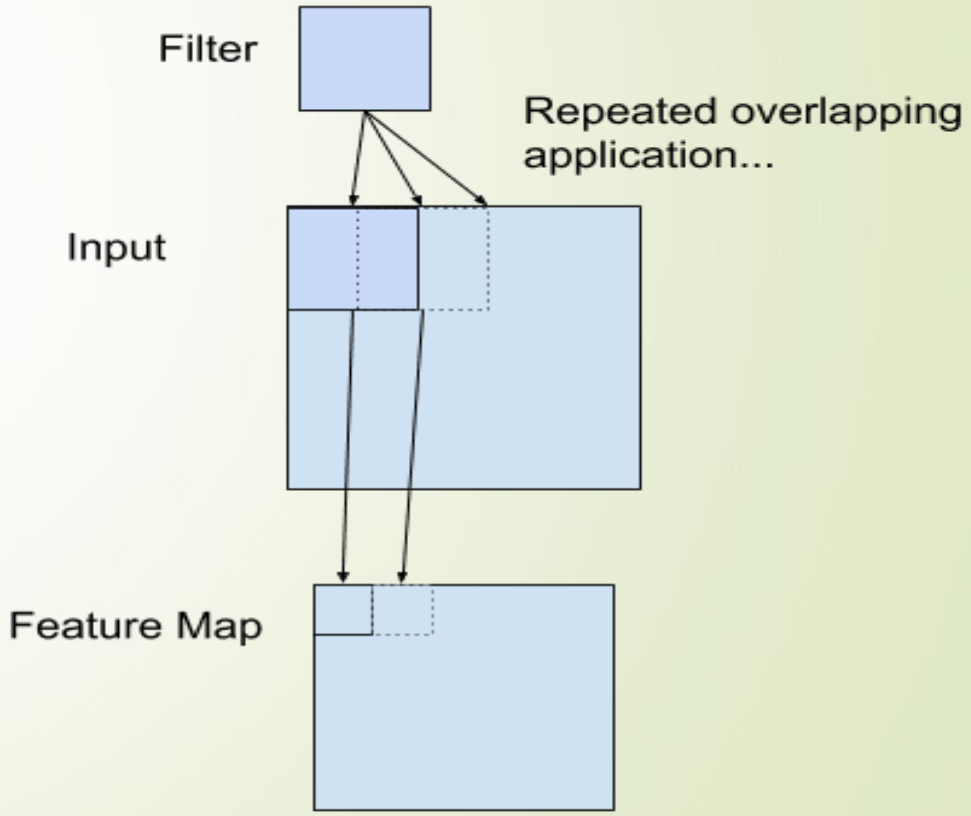
- A convolution tool that **separates and identifies** the various **features of the image** for analysis in a process called as Feature Extraction.

- A fully connected layer that utilizes the output from the convolution process and **predicts the class of the image** based on the features extracted in previous stages.

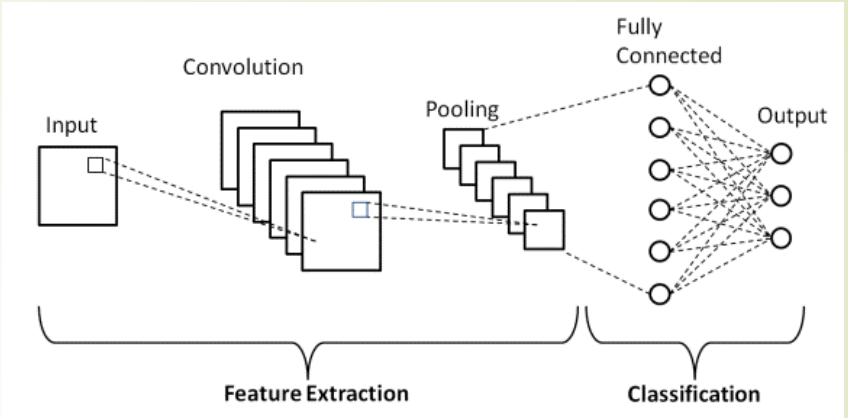
convolutional neural networks

- The convolutional neural network, or CNN for short, is a specialized type of neural network model designed for working with two-dimensional image data, although they can be used with one-dimensional and three-dimensional data.
- Central to the convolutional neural network is the convolutional layer that gives the network its name. This layer performs an operation called a "*convolution*".
- In the context of a convolutional neural network, a convolution is a linear operation that involves the multiplication of a set of weights with the input, much like a traditional neural network.
- Given that the technique was designed for two-dimensional input, the multiplication is performed between an array of input data and a two-dimensional array of weights, called a filter or a kernel.
- The output from multiplying the filter with the input array one time is a single value. As the filter is applied multiple times to the input array, the result is a two-dimensional array of output values that represent a filtering of the input. As such, the two-dimensional output array from this operation is called a "*feature map*".

- **Once a feature map is created, we can pass each value in the feature map through a nonlinearity, such as a ReLU.**
- In summary, we have an ***input***, such as an **image of pixel values, and** we have a ***filter*, which is a set of weights**, and the filter is systematically applied to the input data to create a ***feature map*.**



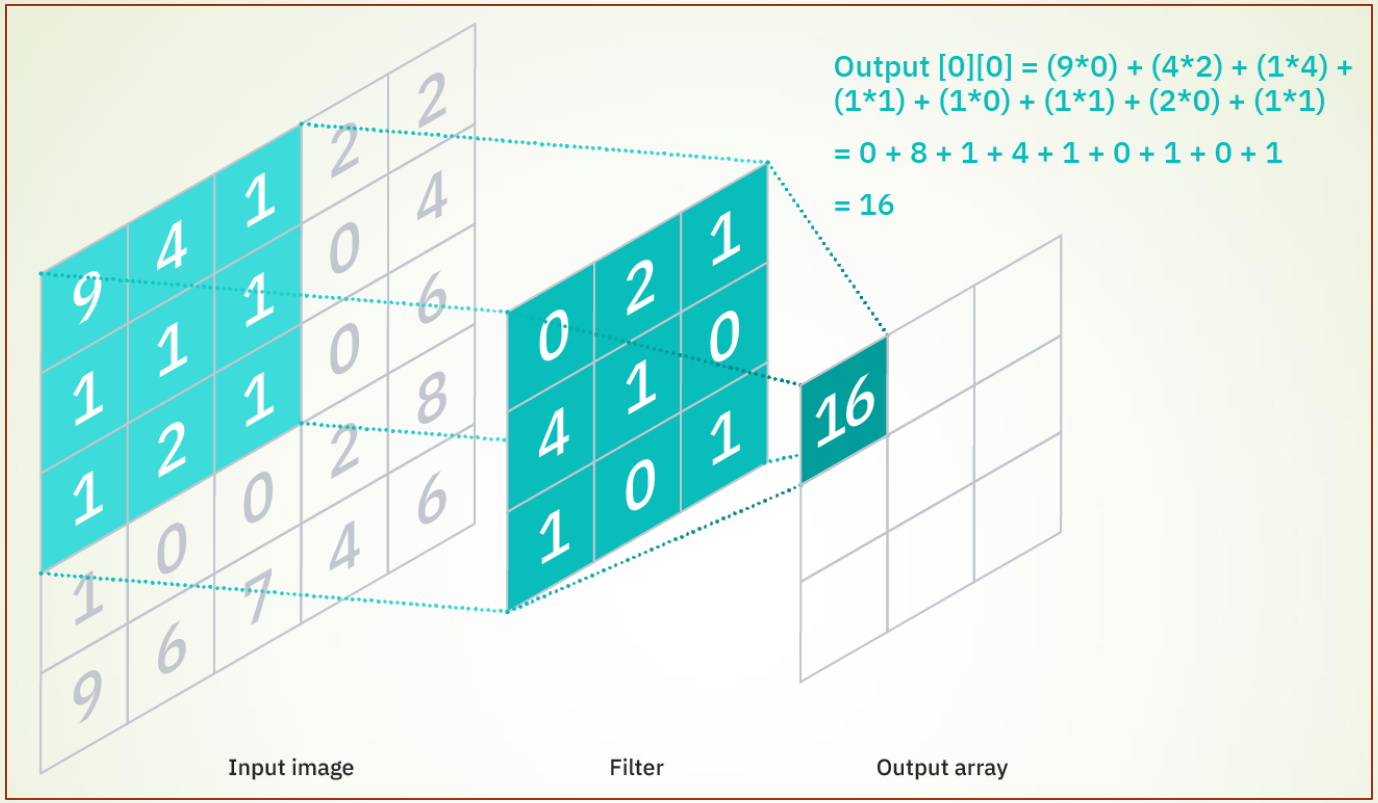
Architecture

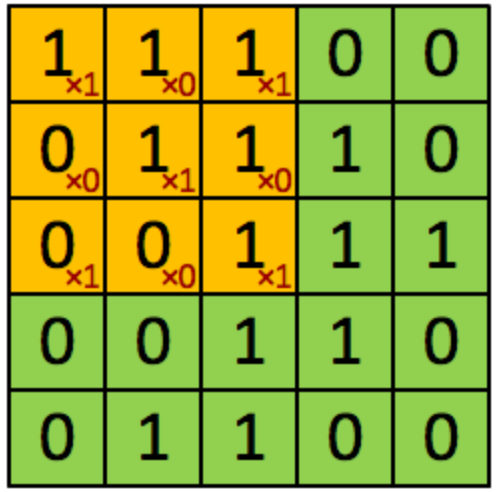


- CNN have three main types of layers, which are:
- Convolutional layer
- Pooling layer
- Fully-connected (FC) layer
- The convolutional layer is the first layer of a convolutional network. While convolutional layers can be followed by additional convolutional layers or pooling layers, the fully-connected layer is the final layer.
- With each layer, the CNN increases in its complexity, identifying greater portions of the image. Earlier layers focus on simple features, such as colors and edges. As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until it finally identifies the intended object.

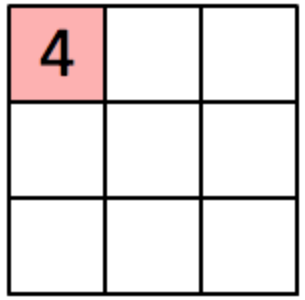
Convolutional Layer

- The convolutional layer is the core building block of a CNN, and it is where the majority of computation occurs. It requires a few components, which are input data, a filter, and a feature map.
- Let's assume that the input will be a color image, which is made up of a matrix of pixels in 3D. This means that the input will have three dimensions—a height, width, and depth—which correspond to RGB in an image. We also have a feature detector, also known as a kernel or a filter, which will move across the receptive fields of the image, checking if the feature is present. This process is known as a convolution.
- The filter size is typically a 3x3 matrix; this also determines the size of the receptive field. The filter is then applied to an area of the image, and a dot product is calculated between the input pixels and the filter. This dot product is then fed into an output array. Afterwards, the filter shifts by a stride, repeating the process until the kernel has swept across the entire image. The final output from the series of dot products from the input and the filter is known as a feature map, activation map, or a convolved feature


Output [0][0] = (9*0) + (4*2) + (1*4) + (1*1) + (1*0) + (1*1) + (2*0) + (1*1)
= 0 + 8 + 1 + 4 + 1 + 0 + 1 + 0 + 1
= 16
16
Input image
Filter
Output array

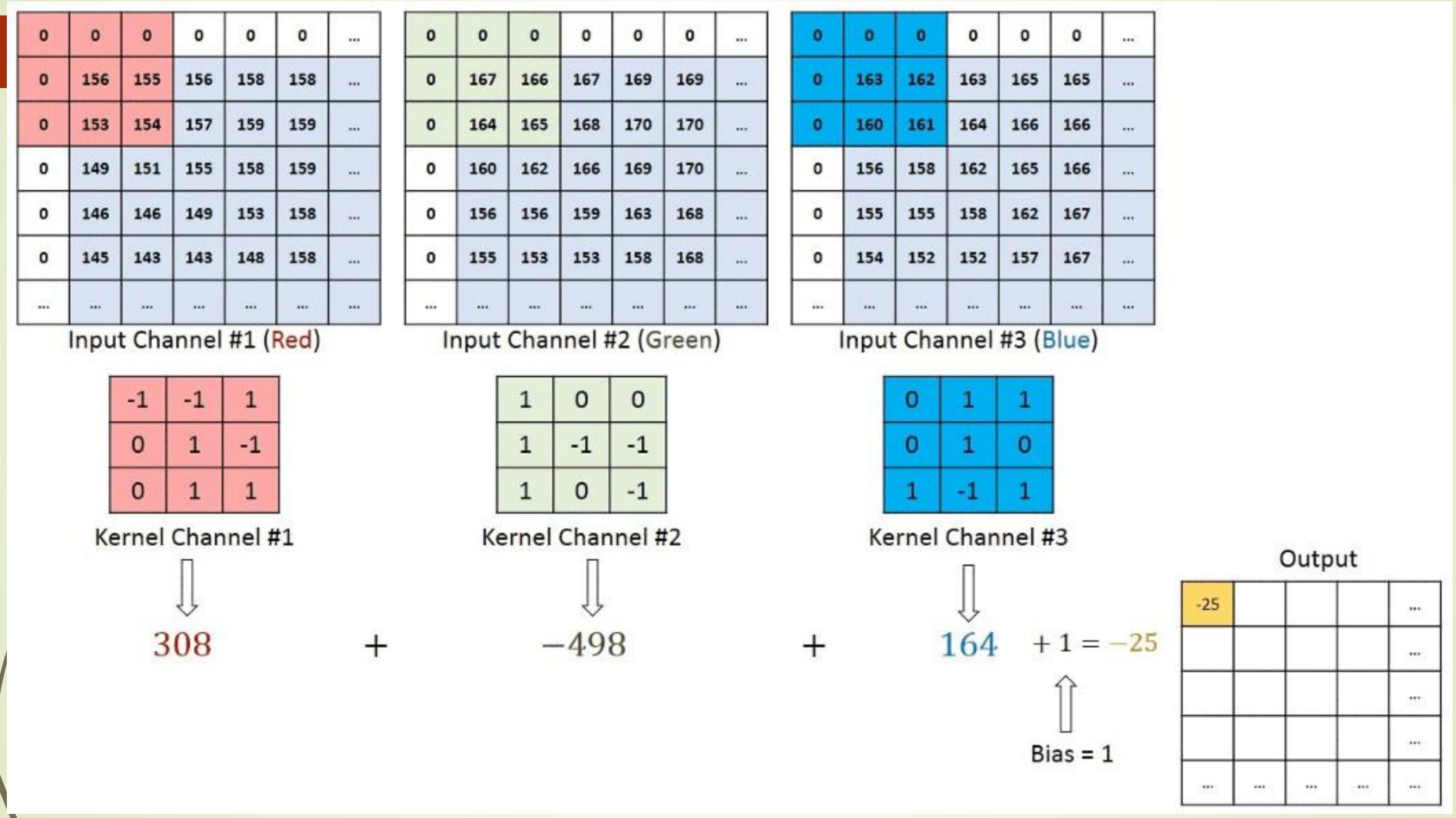


Image



Convolved Feature

Kernel/Filter, K =
1 0 1
0 1 0
1 0 1


0 0 0 0 0 0 ...
0 156 155 156 158 158 ...
0 153 154 157 159 159 ...
0 149 151 155 158 159 ...
0 146 146 149 153 158 ...
0 145 143 143 148 158 ...
... ...
Input Channel #1 (Red)
0 0 0 0 0 0 ...
0 167 166 167 169 169 ...
0 164 165 168 170 170 ...
0 160 162 166 169 170 ...
0 156 156 159 163 168 ...
0 155 153 153 158 168 ...
... ...
Input Channel #2 (Green)
0 0 0 0 0 0 ...
0 163 162 163 165 165 ...
0 160 161 164 166 166 ...
0 156 158 162 165 166 ...
0 155 155 158 162 167 ...
0 154 152 152 157 167 ...
... ...
Input Channel #3 (Blue)
-1 -1 1
0 1 -1
0 1 1
Kernel Channel #1
1 0 0
1 -1 -1
1 0 -1
Kernel Channel #2
0 1 1
0 1 0
1 -1 1
Kernel Channel #3
308
+
−498
+
164
+ 1 = −25
Bias = 1
Output
-25 ... ...
... ...
... ...
... ...
... ...

- Note that the weights in the feature detector remain fixed as it moves across the image, which is also known as *parameter sharing*.

- Some parameters, like the weight values, adjust during training through the process of backpropagation and gradient descent. **However, there are three hyperparameters which affect the volume size of the output that need to be set before the training of the neural network begins.**
- **These include:**

1. **The number of filters affects the depth of the output. For example, three distinct filters would yield three different feature maps, creating a depth of three.**

2. **Stride is the distance, or number of pixels, that the kernel moves over the input matrix. While stride values of two or greater is rare, a larger stride yields a smaller output.**

3. **Padding is usually used when the filters do not fit the input image. This sets all elements that fall outside of the input matrix to zero, producing a larger or equally sized output.**

- You might already notice that the pixels of the image aren't processed with the same number. The pixels at the corner are less counted than those in the middle. This means that the pixels don't get the same amount of weights. Additionally, If we just keep applying the convolution, we might lose the data too fast. ***Padding*** is the trick we can use here to fix this problem. As its name, padding means giving additional pixels at the boundary of the data.
- There are three types of padding:
- **Valid padding:** This is also known as no padding. In this case, the last convolution is dropped if dimensions do not align.
- **Same padding:** This padding ensures that the output layer has the same size as the input layer
- **Full padding:** This type of padding increases the size of the output by adding zeros to the border of the input.
- After each convolution operation, a CNN applies a Rectified Linear Unit (ReLU) transformation to the feature map, introducing nonlinearity to the model.

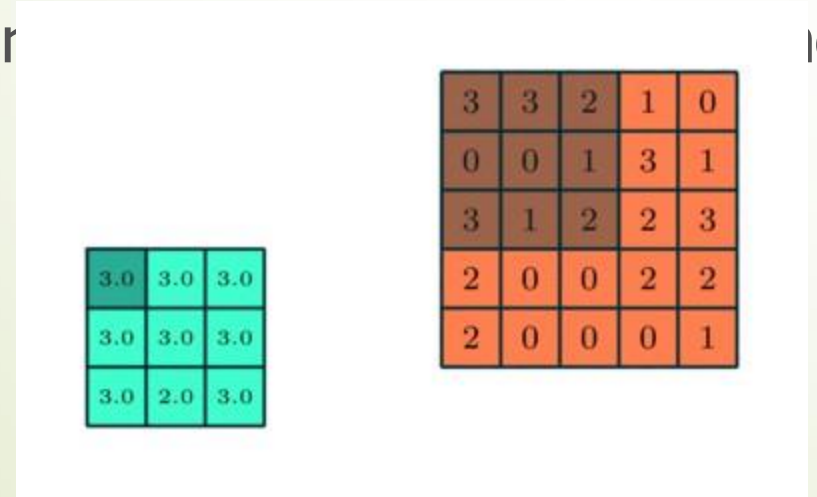
- Then we can ask one question here. Suppose there are a **N** x **N** input image and **Nc`** number of **f** x **f** filters. We'll make **p** number of padding layers and move the filters with the **S** value of stride. Please take a moment to think and converge all we discussed so far. What will be the size of the result?

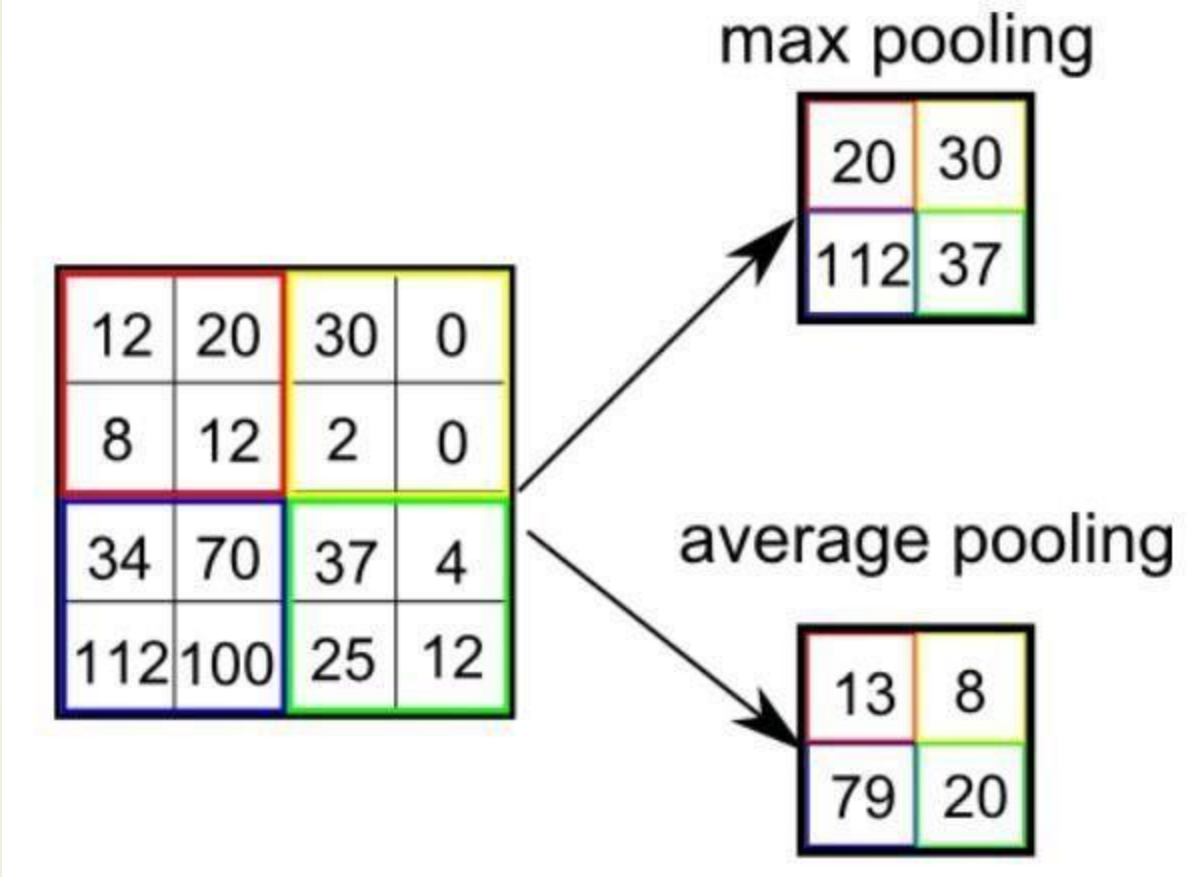
$$\left(\frac{n + 2p - f}{s} + 1\right) \times \left(\frac{n + 2p - f}{s} + 1\right) \times Nc$$

- 7 x 7 x 3 input image
- 3 x 3 x 4 filters
- No padding
- Stride 1
- What will be the size of output image?

Pooling Layer

- Pooling layers, also known as downsampling, conducts dimensionality reduction, reducing the number of parameters in the input.
- **Similar to the convolutional layer, the pooling operation sweeps a filter across the entire input, but the difference is that this filter does not have any weights.**
- Instead, the kernel applies an aggregation function to the values within the r e output array.




max pooling
20 30
112 37
12 20 30 0
8 12 2 0
34 70 37 4
112 100 25 12
average pooling
13 8
79 20

- This is to **decrease the computational power required to process the data through** dimensionality reduction.
- Furthermore, it is useful for **extracting dominant features** which are rotational and positional invariant, thus maintaining the process of effectively training of the model.
- There are two main types of pooling:

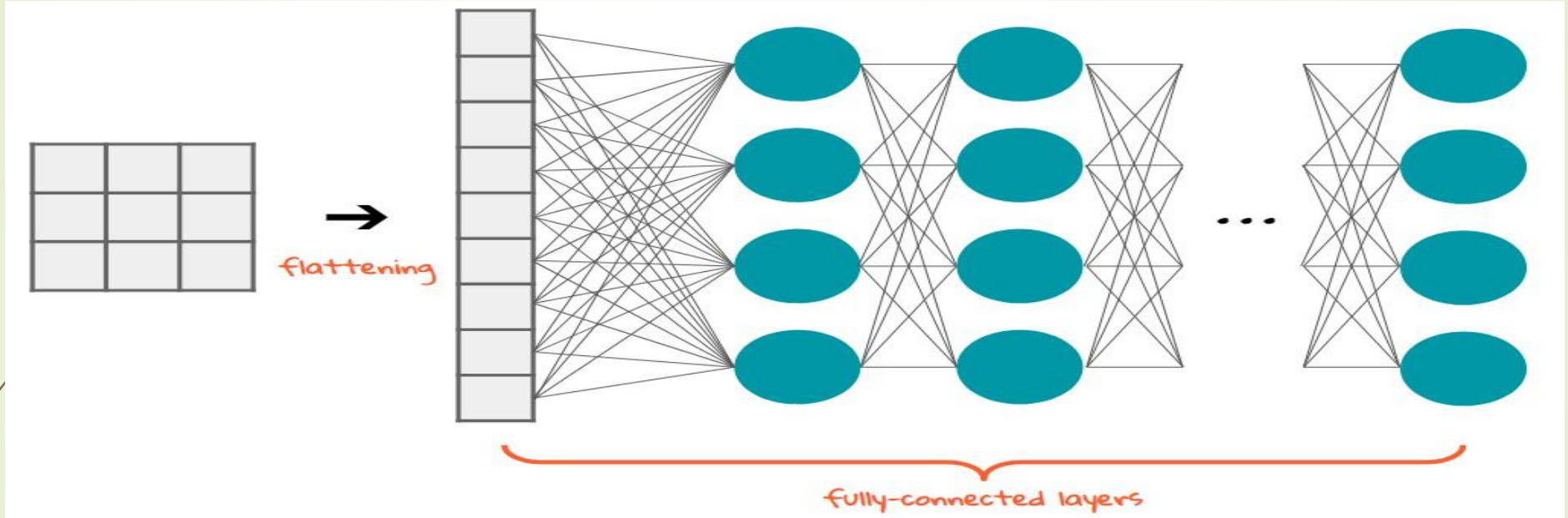
- **Max pooling:** As the filter moves across the input, it selects the pixel with the maximum value to send to the output array. As an aside, this approach tends to be used more often compared to average pooling.
- **Average pooling:** As the filter moves across the input, it calculates the average value within the receptive field to send to the output array.

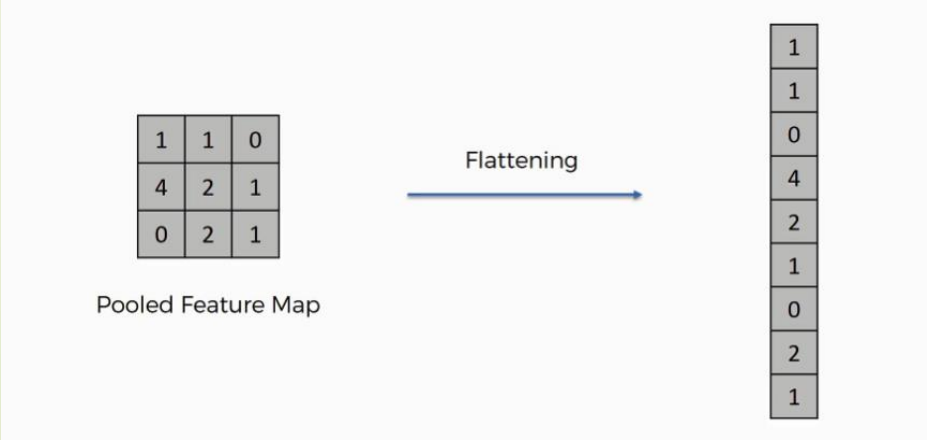
- While a lot of information is lost in the pooling layer, it also has a number of benefits to the CNN. They help to reduce complexity, improve efficiency, and limit risk of overfitting.

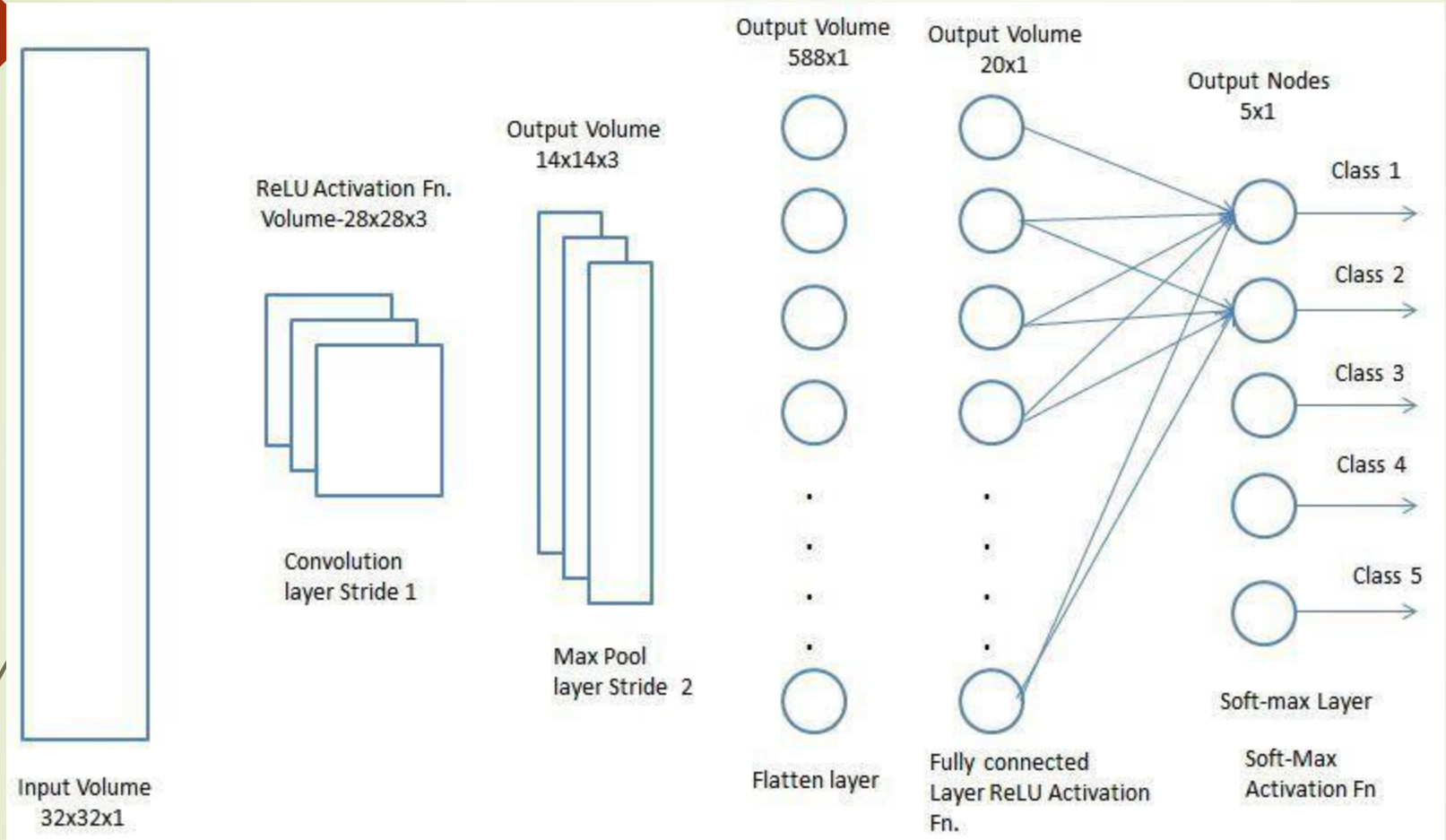
Fully-Connected Layer

- After going through the above process, we have successfully enabled the mod elto understand the features. Moving on, we are going to flatten the final outp ut andfeed it to a regular Neural Network for classification purposes.
- Adding a Fully Connected layer is a (usually) cheap way of learning nonlinear combinations of the high
  level features as represented by the output of the convolutional layer.
- The Fully-Connected layer is learning a possibly non-linear function in that
- space

Flattening is converting the data into a 1-dimensional array for inputting it to the next layer. We flatten the output of the convolutional layers to create a single long feature vector. And it is connected to the final classification model, which is called a ***fully-connected*** layer.






Input Volume
32x32x1
ReLU Activation Fn.
Volume-28x28x3
Convolution
layer Stride 1
Output Volume
14x14x3
Max Pool
layer Stride 2
Output Volume
588x1
Flatten layer
Output Volume
20x1
Fully connected
Layer ReLU Activation
Fn.
Output Nodes
5x1
Class 1
Class 2
Class 3
Class 4
Class 5
Soft-max Layer
Soft-Max
Activation Fn

- Now that we have converted our input image into a suitable form for our Multi-Level Perceptron, we shall flatten the image into a column vector.
- The flattened output is fed to a feed-forward neural network and backpropagation applied to every iteration of training.
- Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using the **Softmax Classification** technique.
- While convolutional and pooling layers tend to use ReLu functions, FC layers usually leverage a softmax activation function to classify inputs appropriately, producing a probability from 0 to 1.